RNN Forward Propagation Example: Predicting 's' from 'dog'

Setup

Task: Predict the next letter 's' given the sequence 'd', 'o', 'g'

Architecture:

- Input layer: One-hot encoded letters
- Hidden layer: 3 nodes with Tanh activation
- Output layer: Softmax activation over vocabulary

Vocabulary: ['d', 'o', 'g', 's'] (simplified for this example)

One-Hot Encoding

- 'd' = [1, 0, 0, 0]
- 'o' = [0, 1, 0, 0]
- 'g' = [0, 0, 1, 0]
- 's' = [0, 0, 0, 1]

Weight Matrices (Example Values)

Input to Hidden (Wxh): 3×4 matrix

```
Wxh = [[ 0.5, -0.2,  0.1,  0.3],
       [-0.1,  0.4,  0.2, -0.3],
       [ 0.2, -0.1,  0.5,  0.1]]
```

Hidden to Hidden (Whh): 3×3 matrix

```
Whh = [[ 0.3, -0.1,  0.2],
       [ 0.1,  0.4, -0.2],
       [-0.2,  0.1,  0.3]]
```

Hidden to Output (Why): 4×3 matrix

```
Why = [[ 0.2,  0.1, -0.3],
       [-0.1,  0.4,  0.2],
       [ 0.3, -0.2,  0.1],
       [ 0.1,  0.3, -0.1]]
```

Bias vectors:

- bh = [0.1, -0.1, 0.2] (hidden bias)
- by = [0, 0, 0, 0] (output bias)

Forward Propagation Steps

Time Step 1: Input 'd'

Input: x_1 = [1, 0, 0, 0] **Previous hidden state**: h_0 = [0, 0, 0] (initialized to zeros)

Hidden layer computation:

```
z₁ = Wxh @ x₁ + Whh @ h₀ + bh
z₁ = [[0.5, -0.2, 0.1, 0.3],     [[1],     [[0.3, -0.1, 0.2],     [[0],     [[0.1],
      [-0.1, 0.4, 0.2, -0.3],  @  [0],  +   [0.1, 0.4, -0.2],  @  [0],  +  [-0.1],
      [0.2, -0.1, 0.5, 0.1]]      [0]]      [-0.2, 0.1, 0.3]]      [0]]      [0.2]]
                                  [0]]
```

```
z₁ = [0.5, -0.1, 0.2] + [0, 0, 0] + [0.1, -0.1, 0.2] = [0.6, -0.2, 0.4]
```

```
h₁ = tanh(z₁) = [tanh(0.6), tanh(-0.2), tanh(0.4)] = [0.537, -0.197, 0.380]
```

Time Step 2: Input 'o'

Input: x_2 = [0, 1, 0, 0] **Previous hidden state**: h_1 = [0.537, -0.197, 0.380]

Hidden layer computation:

```
z₂ = Wxh @ x₂ + Whh @ h₁ + bh
z₂ = [[-0.2], + [[0.3, -0.1, 0.2],     [[0.537],  + [[0.1],
      [0.4],     [0.1, 0.4, -0.2],  @   [-0.197],    [-0.1],
      [-0.1]]    [-0.2, 0.1, 0.3]]      [0.380]]     [0.2]]
```

```
Whh @ h₁ = [0.3×0.537 + (-0.1)×(-0.197) + 0.2×0.380] = [0.257]
           [0.1×0.537 + 0.4×(-0.197) + (-0.2)×0.380]   [-0.078]
           [(-0.2)×0.537 + 0.1×(-0.197) + 0.3×0.380]   [0.017]
```

```
z₂ = [-0.2, 0.4, -0.1] + [0.257, -0.078, 0.017] + [0.1, -0.1, 0.2]
z₂ = [0.157, 0.222, 0.117]
```

```
h₂ = tanh(z₂) = [0.156, 0.218, 0.117]
```

Time Step 3: Input 'g'

Input: x_3 = [0, 0, 1, 0] **Previous hidden state**: h_2 = [0.156, 0.218, 0.117]

Hidden layer computation:

```
z₃ = Wxh @ x₃ + Whh @ h₂ + bh
z₃ = [[0.1], + [[0.3, -0.1, 0.2],     [[0.156],  + [[0.1],
      [0.2],     [0.1, 0.4, -0.2],  @   [0.218],     [-0.1],
      [0.5]]     [-0.2, 0.1, 0.3]]      [0.117]]     [0.2]]

Whh @ h₂ = [0.3×0.156 + (-0.1)×0.218 + 0.2×0.117] = [0.048]
           [0.1×0.156 + 0.4×0.218 + (-0.2)×0.117]   [0.079]
           [(-0.2)×0.156 + 0.1×0.218 + 0.3×0.117]   [0.022]

z₃ = [0.1, 0.2, 0.5] + [0.048, 0.079, 0.022] + [0.1, -0.1, 0.2]
z₃ = [0.248, 0.179, 0.722]

h₃ = tanh(z₃) = [0.243, 0.177, 0.619]
```

Output Computation (Predicting next letter)

Using the final hidden state h_3 to predict the next letter:

```
y = Why @ h₃ + by
y = [[0.2, 0.1, -0.3],     [[0.243],     [[0],
     [-0.1, 0.4, 0.2],  @   [0.177],  +   [0],
     [0.3, -0.2, 0.1],      [0.619]]      [0],
     [0.1, 0.3, -0.1]]                    [0]]

y = [0.2×0.243 + 0.1×0.177 + (-0.3)×0.619] = [-0.119]
    [(-0.1)×0.243 + 0.4×0.177 + 0.2×0.619]   [0.171]
    [0.3×0.243 + (-0.2)×0.177 + 0.1×0.619]   [0.099]
    [0.1×0.243 + 0.3×0.177 + (-0.1)×0.619]   [0.015]
```

Softmax activation:

```
exp(y) = [exp(-0.119), exp(0.171), exp(0.099), exp(0.015)]
       = [0.888, 1.186, 1.104, 1.015]

sum = 0.888 + 1.186 + 1.104 + 1.015 = 4.193

Probabilities:
P('d') = 0.888/4.193 = 0.212
P('o') = 1.186/4.193 = 0.283
P('g') = 1.104/4.193 = 0.263
P('s') = 1.015/4.193 = 0.242
```

Result

The RNN predicts the next letter with probabilities:

- 'd': 21.2%
- 'o': 28.3% (highest probability)
- 'g': 26.3%
- 's': 24.2%

In this example, the model incorrectly predicts 'o' as the most likely next letter instead of 's'. This demonstrates that the randomly initialized weights don't capture the pattern yet - the model would need training to learn that 's' should follow 'dog'.

Key Observations

1. **Sequential processing**: Each time step uses information from previous hidden states
2. **Information flow**: The hidden state h_3 contains information about the entire sequence 'd', 'o', 'g'
3. **Memory**: The recurrent connections allow the network to "remember" previous inputs
4. **Training needed**: Random weights produce poor predictions - the model needs training with backpropagation through time (BPTT)